

Decorator

Case Study

บริษัทพัฒนาซอฟต์แวร์ได้รับมอบหมายให้ออกแบบระบบสั่งกาแฟสำหรับร้านกาแฟแห่งหนึ่ง โดยลูกค้าสามารถเลือกเครื่องดื่มหลักและเพิ่มส่วนประกอบเสริมได้อย่างอิสระ

Problem Analysis

1. หากใช้ Inheritance เพียงอย่างเดียว จะเกิดปัญหาอะไรขึ้นเมื่อมีการเพิ่ม Add-on จำนวนมาก?

ตอบ: หากใช้ Inheritance จะต้องสร้างคลาสสำหรับการผสมของกาแฟและ Add-on ทำให้จำนวนคลาสเพิ่มขึ้นเยอะมาก โค้ดซับซ้อน และดูแลรักษายาก

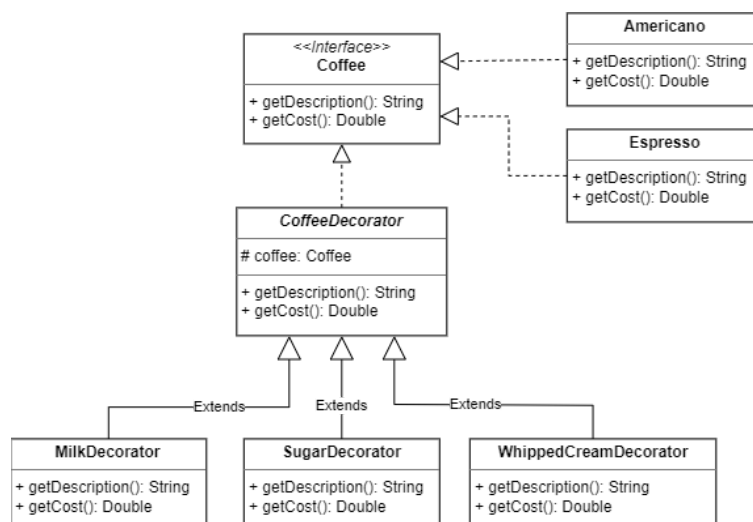
2. เพราะเหตุใด Decorator Pattern จึงเหมาะสมกับระบบนี้?

ตอบ: เพราะสามารถเพิ่ม Add-on ให้กับกาแฟได้อย่างยืดหยุ่น และซับซ้อนหลายชั้นได้ โดยไม่ต้องสร้างคลาสใหม่สำหรับทุก Combination

3. Decorator Pattern สอดคล้อง Open-Closed Principle อย่างไร

ตอบ: Decorator Pattern ช่วยให้สามารถเพิ่ม Add-on ใหม่ได้โดยสร้าง Decorator คลาสใหม่ โดยไม่ต้องแก้ไขคลาสเดิม ทำให้ระบบขยายความสามารถได้ง่าย และเป็นไปตามหลัก OCP

UML Design



Design Reflection

1. Decorator Pattern แตกต่างจาก Strategy Pattern อย่างไร?

ตอบ: Decorator ใช้เพิ่มความสามารถให้ Object แบบยืดหยุ่น ส่วน Strategy ใช้เลือกวิธีการทำงานหรือ Algorithm ที่แตกต่างกัน

2. หากมี Add-on ใหม่เพิ่มขึ้น 20 รายการ ระบบที่ใช้ Decorator Pattern จะมีข้อดีอย่างไร?

ตอบ: สามารถสร้าง Decorator ใหม่เพิ่มได้โดยไม่ต้องแก้ไขคลาสเดิม และสามารถนำ Add-on มาผสมกันได้อย่างอิสระ

3. หลักการ "Composition over Inheritance" ปรากฏอยู่ในส่วนใดของงานนี้

ตอบ: ปรากฏใน CoffeeDecorator ที่เก็บและใช้ Object ของ Coffee แทนการสร้างคลาสย่อยสำหรับทุก Combination ของ Add-on

4. จงยกตัวอย่างระบบจริงที่สามารถใช้ Decorator Pattern ได้อีกอย่างน้อย 2 ระบบ?

ตอบ: ระบบสั่งอาหาร เช่น Burger + Cheese + Bacon + Sauce และระบบแจ้งเตือน เช่น Notification + Email + SMS + Push Notification

Source Code

Coffee.java

```
Java
public interface Coffee {
    public String getDescription();
    public double getCost();
}
```

Espresso.java

```
Java
public class Espresso implements Coffee{
    @Override
    public String getDescription() {
        return "Espresso";
    }

    @Override
```

```
    public double getCost() {  
        return 2.00;  
    }  
}
```

Americano.java

Java

```
public class Americano implements Coffee {  
    @Override  
    public String getDescription() {  
        return "Americano";  
    }  
  
    @Override  
    public double getCost() {  
        return 2.50;  
    }  
}
```

CoffeeDecorator.java

Java

```
public abstract class CoffeeDecorator implements Coffee {  
    protected Coffee coffee;  
  
    public CoffeeDecorator(Coffee coffee) {  
        this.coffee = coffee;  
    }  
  
    @Override  
    public String getDescription() {  
        return coffee.getDescription();  
    }  
  
    @Override  
    public double getCost() {  
        return coffee.getCost();  
    }  
}
```

MilkDecorator.java

Java

```
public class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + " + Milk";
    }

    @Override
    public double getCost() {
        return coffee.getCost() + 0.50;
    }
}
```

SugarDecorator.java

Java

```
public class SugarDecorator extends CoffeeDecorator {
    public SugarDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + " + Sugar";
    }

    @Override
    public double getCost() {
        return coffee.getCost() + 0.20;
    }
}
```

WhippedCreamDecorator.java

Java

```
public class WhippedCreamDecorator extends CoffeeDecorator {
    public WhippedCreamDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + " + Whipped Cream";
    }
}
```

```

@Override
public double getCost() {
    return coffee.getCost() + 0.70;
}
}

```

main.java

```

Java
public class main {
    public static void main(String[] args) throws Exception {
        // Test Case 1
        Coffee order1 = new Espresso();
        System.out.println("Test Case 1");
        System.out.println("Configuration: Espresso");
        System.out.println("Description: " + order1.getDescription());
        System.out.printf("Total Cost: $%.2f\n\n", order1.getCost());

        // Test Case 2
        Coffee order2 = new MilkDecorator(new Espresso());
        System.out.println("Test Case 2");
        System.out.println("Configuration: Espresso + Milk");
        System.out.println("Description: " + order2.getDescription());
        System.out.printf("Total Cost: $%.2f\n\n", order2.getCost());

        // Test Case 3
        Coffee order3 = new WhippedCreamDecorator(new SugarDecorator(new
MilkDecorator(new Americano())));
        System.out.println("Test Case 3");
        System.out.println("Configuration: Americano + Milk + Sugar + Whipped
Cream");
        System.out.println("Description: " + order3.getDescription());
        System.out.printf("Total Cost: $%.2f\n", order3.getCost());
    }
}

```

Output

```

Test Case 1
Configuration: Espresso
Description: Espresso
Total Cost: $2.00

Test Case 2
Configuration: Espresso + Milk
Description: Espresso + Milk
Total Cost: $2.50

Test Case 3
Configuration: Americano + Milk + Sugar + Whipped Cream
Description: Americano + Milk + Sugar + Whipped Cream
Total Cost: $3.90

```